DETECTION OF CIPHER TYPES USING ARTIFICIAL NEURAL NETWORKS

Malay Kule[1], Krishnendu Bera[1], Abhiroop Mukherjee[1], Arnab Sen[1], Swarnali Mondal[1], Sanjana Chakravarty[1], and Rajdeep Ghosh[1]

Indian Institute of Engineering Science and Technology, Shibpur
malay.kule@gmail.com

Abstract. The identification of a cryptosystem has been a challenge for decades. In this paper, the main objective is to identify the type of cryptosystem used to encrypt a certain piece of text. We have explored the realm of Artificial Neural Networks(ANNs) to recognize a pattern among complex classical cipher texts that generally have a simple representation in plain text. We have modelled our objective as a sequence-to-sequence learning task that we have tried to solve using Convolution Neural Networks (CNNs) and state-of-the-art Transformer models. With only a small dataset (130k) consisting of cipher texts and the corresponding cryptosystem used to encrypt the same, our model has shown a promising accuracy of 96.72 % which proves a significantly steep learning curve as compared to other sequence-to-sequence models. Here we show the enormous potential of these models and how they can perform even better if the barrier of resources and computation time is lifted.

Keywords: Artificial Neural Networks(ANNs)· Cryptosystem Identification · Classical Cipher text classification · Convolution Neural Networks(CNNs) · Transformer models.

1 Introduction

In today's world, information is a powerful tool. However, it is essential to protect information and data from unauthorized access, distortion, or any other unauthorized activity. Cryptography is the study of secret writing and is used to prevent unauthorized access to data. It works by using a key(s) to convert plaintext messages into unreadable ciphertext and vice versa. Encryption is the process of converting plaintext message into an unreadable format to protect sensitive information (called ciphertext), while the inverse process is called decryption. Only authorized users can encrypt or decrypt messages [1].

The goal of the work is to use artificial neural networks to examine encrypted text files and classify them based on their cipher types. The process involves creating a collection of cipher texts with various cipher types, building a Deep Neural Network (DNN), and training it with a variety of cipher texts and training parameters to enable accurate learning and cipher text classification [2]. Once the DNN is trained, it can be used to determine the cipher text's kind. In this

work, the Caesar, Affine, Vigenère, Substitution, and Rail Fence ciphers were used to encrypt English texts. The encrypted files were then submitted to CNN and Transformer models, which made predictions, and the accuracy of these predictions was assessed.

2 Preliminaries

This paper requires basic knowledge of neural networks and deep learning, as well as an overview of the encryption algorithms used. These algorithms can be useful for feature extraction. The main machine-learning topics covered are artificial neural networks (ANN), perceptrons, recurrent neural networks (RNN), convolutional neural networks (CNN), and transformers.

- **Artificial Neural Network (ANN)** – An artificial neural network (ANN) is a type of computational model that mimics the human brain structurally and functionally [3]. ANNs consist of layers of interconnected neurons, which process input data and generate output values. ANNs are trained to perform numerous types of tasks, such as classification, regression, and clustering.
- **Perceptron** – A perceptron is a type of artificial neural network (ANN) that can be used for binary classification tasks [4]. It consists of a single layer of neurons, with each neuron receiving input from multiple sources, and producing a single output value. A set of weights determines the output of a perceptron. Those weights are then applied to the input values, and a threshold value is used to determine whether the output is 0 or 1.
- **Convolutional Neural Network (CNN)** – CNNs consist of multiple layers of interconnected neurons, which process the input data in a hierarchical manner [5]. The first layer of a CNN typically consists of a set of filters. These filters are used extract features such as edges and textures by applying them to the input data. These features are then passed to the next layer of the CNN, where they are combined and processed to extract more complex features, such as shapes and objects. The output of a CNN is a set of probabilities that indicate the likelihood that the input data belongs to each of the possible classes.
- **Transformers** – The transformer is a type of encoder-decoder model, which is commonly used for tasks such as machine translation and text summarization [6]. The key innovation of the transformer is the use of self-attention, a mechanism that allows the model to learn dependencies between different words in the input sentence. This allows the transformer to process the input sentence in parallel, rather than in a sequential manner, which makes it much more efficient and scalable than previous NLP models.
- **Attention Mechanism** – Self-attention is a the mechanism used in the transformer [6].With self-attention the model learns dependencies between different words in the input, by computing weighted sum of the input words based on their relevance to each other.

3 Proposed Work

3.1 Dataset Generation

The dataset has been generated for five types of ciphers:

- Caesar
- Affine
- Vigenère
- Substitution
- Rail Fence

The plaintext used for encryption was the texts from Shakespeare's Hamlet [7], Macbeth [8], Merchant of Venice [9] and Romeo and Juliet [10].

We converted the text to lowercase and removed all whitespaces and punctuation. Then the plaintext was encrypted and exported as a CSV.

3.2 CNN Based Proposed Model

The main intuition behind this method is that CNN uses its kernel as a "local receptive field" [5], which means that a CNN kernel (small in size as we are dealing with stream cipher and not block cipher) can look its nearby characters and will find some pattern in the ciphertext.

So, we first converted every ciphertext into their respective list of tokens using a TextVectorizer [11], and then we converted that list into an image (see Fig. 1).

$$\begin{bmatrix} a & b & c & d \end{bmatrix} \longrightarrow \begin{bmatrix} a & b & c & d \\ b & c & d & a \\ c & d & a & b \\ d & a & b & c \end{bmatrix}$$

Fig. 1: Converting text to 2D text matrix

Note that this conversion depends on the length of the string used, i.e. for a string for length n, we will generate a $n \times n$ matrix. After we generate all variable-sized matrices, we resize them to 64×64 using Bilinear Interpolation.

Now we can see that a kernel (say 3×3 with stride one) can observe its previous as well as its next two characters and can use that information to find patterns in the ciphertext.

We use a CNN to extract features from the matrix and then we feed those features to a Dense Network to classify between the ciphertext types [5].

3.3 Motivation Behind Choosing Transformer Model

In our case, we used pre-trained transformer models, which have been fine-tuned using our own dataset. This allowed us to leverage the powerful language understanding capabilities of the transformer architecture while also ensuring that the model was specifically tailored to our problem.

The transformer architecture is based on the idea of self-attention, which allows the model to weigh the importance of different tokens in the input sequence when making predictions [6]. This is done by computing a dot product between the input tokens and a set of learnable parameters, which are then used to generate a weight for each token. These weights are then used to compute a weighted average of the input tokens, which is used as the input to the next layer of the model.

This self-attention mechanism allows the transformer to effectively capture long-range dependencies between tokens, which is crucial for understanding the relationship between the ciphertext and the algorithm used to generate it.

In summary, we have used the transformer model as it is a state-of-the-art model for sequence-to-sequence problems and its self-attention mechanism and multi-head attention allow the model to capture long-range dependencies between tokens and attend to different parts of the input sequence in parallel, which is crucial for understanding the relationship between the cipher-text and the algorithm used to generate it.

3.4 Transformer Based Proposed Model

The Transformer in NLP is a novel architecture that aims to solve sequence-to-sequence tasks while handling long-range dependencies with ease. The Transformer was proposed in the paper Attention Is All You Need [6]. The main intuition behind this method is to compute the relative importance of a ciphertext character with respect to the other ciphertext characters.

Our dataset contains encrypted text and the algorithm used for encryption, we create a text prompt from each row for example:

Cipher text: `ymccaygimgamkyyiycuycqocyeagmigkycmyyoyy`
Algorithm: `affine`

The model learns the relative importance of each token (character) and uses it to predict the type of ciphertext. For example, if the model is trained with the above prompt and then we supply the following prompt:

Cipher text: `yycmdkreqluithnalekstndonwothvn`
Algorithm:

The model may generate the output as “affine” which it learned from training.

In our use case, we have used the GPT-Neo (Generative Pre-Trained) [12], a model with 125 million parameters to achieve our objective. It consists of a set of encoders and decoders placed one after the other. It has been pre-trained

on Wikipedia Corpus [13] and Common Crawl [14] (This includes 12 years of data uploaded on the internet and has a size of over 12 Petabytes). The word "Generative" suggests that the model is capable of text generation. The architecture is an extension to GPT2 [15]. GPT-Neo [12] uses local attention in every other layer with a window size of 256 tokens. All these models are available for free in the Hugging Face [16] transformers library which can be used for a variety of NLP tasks.

The following steps were taken for this model:

1. Each data is converted into text prompts that is passed through a tokenizer that creates embedding for each token in our input string.
2. The input dataset has been split into the ratio 80:10:10 for training, testing, and validating purposes.
3. The training and validation set has been used to fine-tune the model's hyperparameters specific to our use-case i.e. detecting the algorithm used in the ciphertext generation.
4. We used the fine-tuned model on our test dataset to check the accuracy of our model.
5. After fine-tuning, for each of our test data we performed sampling with the temperature set to 0.7 to return 10 predictions. The maximum predicted algorithm is chosen to be the correct one and is then compared with the actual value to compute the accuracy.

For our testing purpose, we used various decoding strategies to find the one which gives the best result. Out of the strategies: Greedy Search, Beam Search and Sampling with set temperature, we found sampling to give the most accurate results of all three. It is a method that is in between the greedy and sampling approach.

The greedy method selects the next token that has the highest probability as the output while sampling selects one of the possible next tokens randomly. When we set a temperature, the randomness is lowered such that the likelihood of high-probability words is increased and that of the low probability is decreased. We found that a temperature set to 0.7 gave the optimum accuracy on our test data. 10 predictions are computed for each test data and the maximum predicted algorithm is chosen to be the correct one and is then compared with the actual value to compute the accuracy.

4 Experimental Results

4.1 Dataset-1

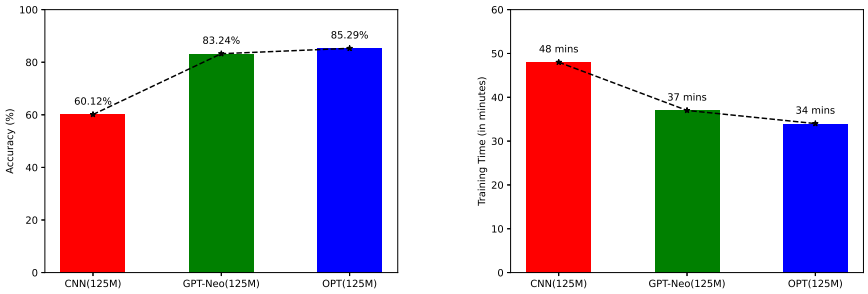
Our initial dataset consisted of 34075 rows that were split for training, testing, and validating in the ratio 80:10:10. The following are the sizes for each split:

- Train dataset: 27260 rows
- Validation dataset: 3408 rows
- Test dataset: 3407 rows

The following are the results obtained:

Table 1: Training time and accuracy on Dataset-1

Model	Number of Classes	Training time	Accuracy
CNN(125M)	5	48 mins	60.12%
GPT-Neo(125M)	5	37 mins	83.24%
OPT(125M)	5	34 mins	85.29%



(a) Accuracy (b) Training Time

Fig. 2: Comparison of Accuracies and Training Times on Dataset 1

4.2 Dataset-2

Next, we increased our dataset containing 133880 rows that were split for training, testing, and validating in the ratio 80:10:10. The following are the sizes for each split:

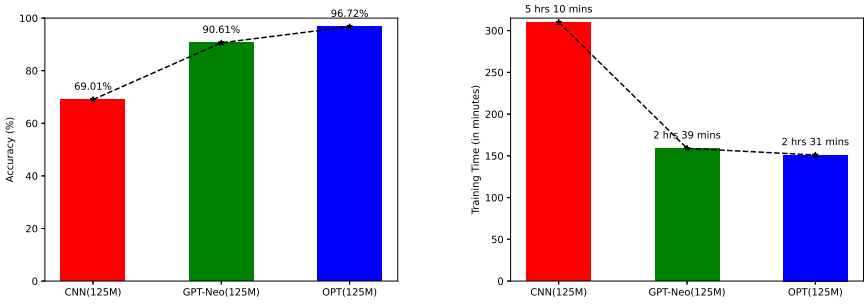
- Train dataset: 107104 rows

– Validation dataset: 13388 rows
– Test dataset: 13388 rows

The following are the results obtained:

Table 2: Training time and accuracy on Dataset-2

Model	Number of Classes	Training time	Accuracy
CNN(125M)	5	5 hrs 10 mins	69.01%
GPT-Neo(125M)	5	2 hrs 39 mins	90.61%
OPT(125M)	5	2 hrs 31 mins	96.72%



(a) Accuracy (b) Training Time

Fig. 3: Comparison of Accuracies and Training Times on Dataset 2

5 Conclusion

This paper shows the current progress of our work in the detection of ciphertext using Artificial Neural Networks. Here we used three types of Neural Networks to achieve our objective, namely: Convolution Neural Networks and GPT-Neo and OPT.

With the CNN approach, our model was able to achieve an accuracy of 68.30% for 5 class classifications. We saw a significant improvement with the transformers approach using GPT-Neo on the same dataset and classes giving an accuracy of 90.61%. This was primarily due to the ability of GPT-Neo to memorize the local importance of tokens due to the self-attention feature. OPT gave better improvements, giving an accuracy of 96.72%, signifying that OPT is better than GPT-Neo for the task of detecting the type of ciphertext. We also

observed that transformer based models are giving better result than CNN while taking less time for training.

In future work, we will use similar transformer based model like BERT, GPTJ and Bloom to see if those models are better suited for the task. We will also extend the number of classes of ciphertext and size of dataset to see if we observe any improvements.

References

1. Katz, J., Lindell, Y.: Introduction to modern cryptography. CRC press (2020)
2. Kopal, N.: Of ciphers and neurons – detecting the type of ciphers using artificial neural networks. In: 3rd International Conference on Historical Cryptology HistoCrypt 2020. pp. 77–86 (05 2020)
3. Zupan, J.: Introduction to artificial neural network (ann) methods: what they are and how to use them. Acta Chimica Slovenica **41**, 327–327 (1994)
4. Gallant, S.I., et al.: Perceptron-based learning algorithms. IEEE Transactions on neural networks **1**(2), 179–191 (1990)
5. O'Shea, K., Nash, R.: An introduction to convolutional neural networks. CoRR **abs/1511.08458** (2015), http://arxiv.org/abs/1511.08458
6. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2017)
7. Shakespeare, W.: 2015/hamlet.txt at master · cs109/2015. https://github.com/cs109/2015/blob/master/Lectures/Lecture15b/sparklect/shakes/hamlet.txt
8. Shakespeare, W.: 2015/macbeth.txt at master · cs109/2015. https://github.com/cs109/2015/blob/master/Lectures/Lecture15b/sparklect/shakes/macbeth.txt
9. Shakespeare, W.: 2015/merchantofvenice.txt at master · cs109/2015. https://github.com/cs109/2015/blob/master/Lectures/Lecture15b/sparklect/shakes/merchantofvenice.txt
10. Shakespeare, W.: 2015/romeojuliet.txt at master · cs109/2015. https://github.com/cs109/2015/blob/master/Lectures/Lecture15b/sparklect/shakes/romeojuliet.txt
11. Tensorflow: tf.keras.layers.textvectorization — tensorflow v2.11.0. https://www.tensorflow.org/api_docs/python/tf/keras/layers/TextVectorization
12. Neo, G.: Gpt neo. https://huggingface.co/docs/transformers/model_doc/gpt_neo
13. English-Corpora: English-corpora: Wikipedia. https://www.english-corpora.org/wiki/
14. Crawl, C.: Common crawl. https://commoncrawl.org/
15. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al.: Language models are unsupervised multitask learners. OpenAI blog **1**(8), 9 (2019)
16. Face, H.: Hugging face – the ai community building the future. https://huggingface.co/